Lab 3：Sv39 分页式虚拟内存系统

3230104220 徐晟洋
2025.11.9

1　从物理地址到虚拟地址

1.1　整体逻辑

1. 分页式虚拟内存，存的是逻辑地址，所以需要页表将其定位到物理地址。
2. 而页表最开始也在内存里，所以需要找到一种找到页表的方法。否则会陷入“蛋生鸡，鸡生蛋”的死循环

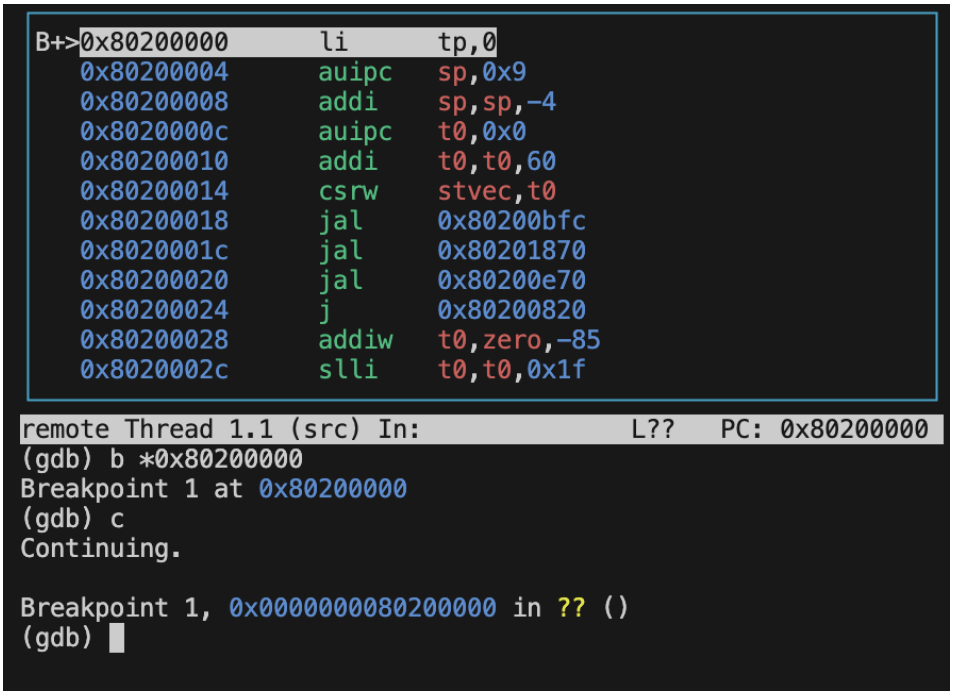
所以流程如下：

1. 内核启动，在物理地址 0x80200000 开始执行
2. 采用PC相对寻址（即逻辑地址作差，然后加上物理地址），执行setup_vm，创建页表
3. 将页表的地址写入satp寄存器，至此，页表加载完成
4. 开启分页，MMU启动，并开始重定位

1.2　动手做：验证内核加载的仍是物理地址

动手调试一下刚编译的内核：

- 在 `0x80200000` 处打断点，让 GDB 运行到断点处停下来；
- 你会发现 GDB 现在没法解析源码位置（比如断点处停下来的时候显示 `0x80200000 in ?? ()`）。这是因为 GDB 是根据符号的位置来推算当前位于哪个函数的，现在所有符号都变成了虚拟地址，它现在找不到 `0x80200000` 对应哪个函数；
- 使用 GDB 或 QEMU Monitor，将从 `0x80200000` 开始的内存打印为指令，与 `vmlinux.asm` 中的反汇编结果对比，它们是否一致？



```
(gdb) x/10xw 0x80200018
0x80200018:     0x3e5000ef      0x055010ef      0x651000ef
  0x7fc0006f
```

```
call mm_init
ffffffd600200018: 3e5000ef            jal ffffffd600200bfc <mm_init>
```

以 `mm_init()` 函数的调用为例

`objdump` 反汇编结果显示 `jal` 指令跳转到 `0xffffffd600200bfc`

GDB 显示跳转到 `0x80200bfc`。但是，两边的机器码都是 `0x3e5000ef`，说明它们实际上是同一条指令。

1.3 动手做：解释同一条指令的不同翻译结果

转换结果：jal x1, 3044

jal指令采用的是PC相对寻址

在vmlinux.asm中，当前的PC地址是0xffffffd600200018

GDB中，当前地址是0x80200018

3044转换为16进制为0xbe4，两者加上0xbe4，地址分别为`0xffffffd600200bfc`和`0x80200bfc`

进而解释了翻译结果不同的原因，本质原因是当前的PC地址不同。vmlinux.asm中为逻辑地址，GDB中为物理地址。

2 构造 Sv39 页表

2.1 前置知识

2.1.1 sv39的逻辑

1. 一个**三级**的查询结构。
2. 地址被切成**三级索引 + 偏移量**。
3. 每一级目录里的条目(PTE)不仅指向下一级，还带了**权限开关**。
4. 有一个叫**大页**的“捷径”，可以用来快速映射大块内存。

2.1.2 翻译过程

2.1.2.1 Sv32

1. 虚拟地址：VPN作为索引，对应到相应的页表

31 — 22	21 — 12	11 — 0
VPN[1]	VPN[0]	page offset
10	10	12

2. 页表
   (a) 对应方式：对应的是物理地址，而非索引！！！

   L1页表项中存储的 PPN 是下一级 L0 页表的**物理页号**，而**不是** L0 页表在一个预设数组中的索引。

(b) 分为两种情况，要区分是指向下一个页表的指针（中间表项），还是指向目标地址了（叶子表项）

(c) 具体的含义

i. v表示这个页表是否valid

ii. 其他详见文档

31	20 19	10 9	8	7	6	5	4	3	2	1	0
PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V
12	10	2	1	1	1	1	1	1	1	1

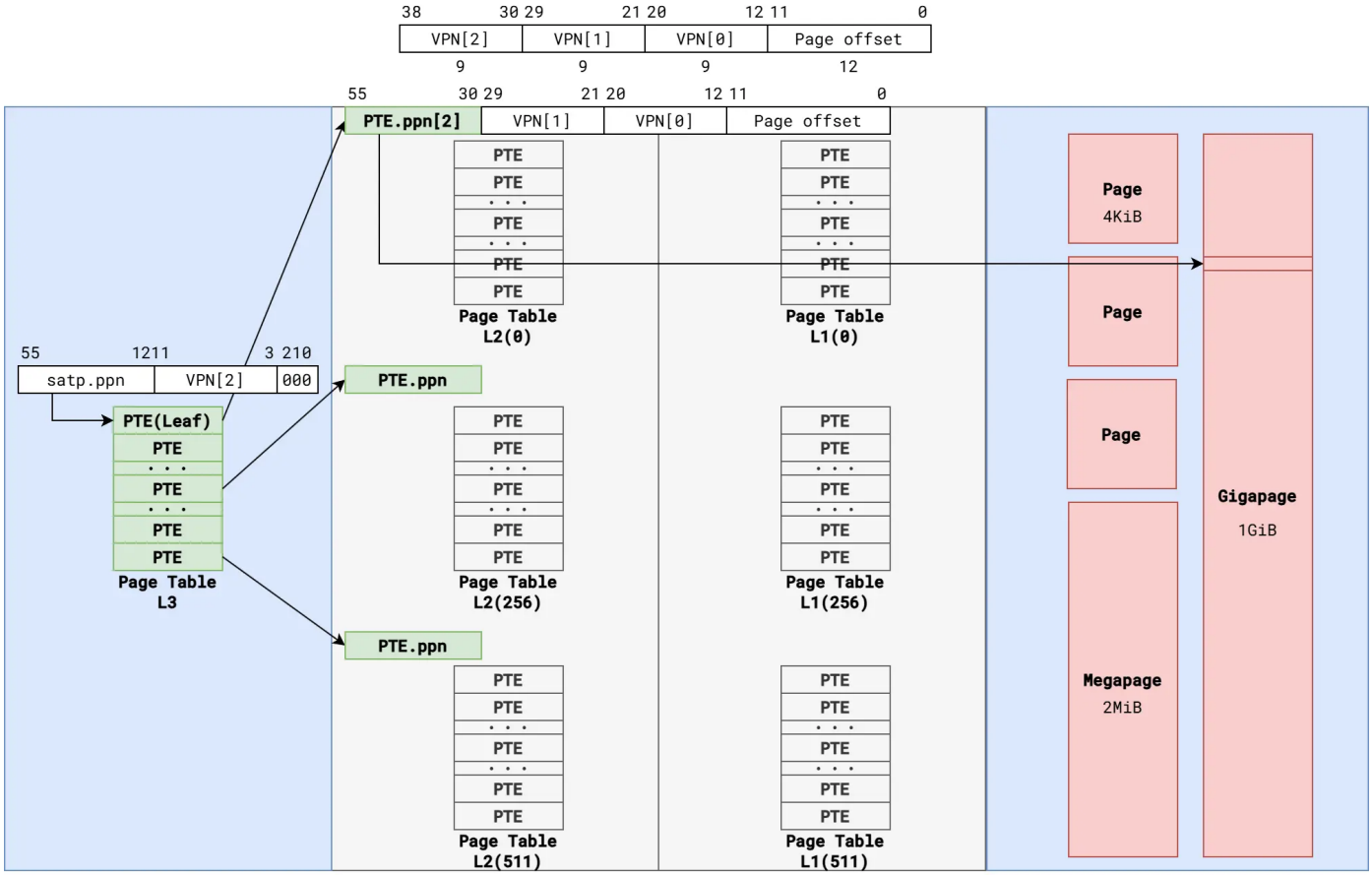
2.1.2.2 Sv39

38 30	29 21	20 12	11 0
VPN[2]	VPN[1]	VPN[0]	page offset
9	9	9	12

63	62 61	60 54	53 28	27 19	18 10	9 8	7	6	5	4	3	2	1	0
N	PBMT	Reserved	PPN[2]	PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V
1	2	7	26	9	9	2	1	1	1	1	1	1	1	1

55 30	29 21	20 12	11 0
PPN[2]	PPN[1]	PPN[0]	page offset
26	9	9	12

对应实例



Sv39における仮想アドレス変換(Gigapage)

2.2 动手做：中断实现重定位

阅读 Linux 内核源码 `arch/riscv/kernel/head.S` 的 `relocate_enable_mmu()` 函数，解释它是怎么实现重定位的。

其采用了自陷阱法

1. 首先预设陷阱，在开启 MMU 之前，将中断向量 (stvec) 设置为目标代码的最终虚拟地址。

```
la a2, 1f
add a2, a2, a1
csrw CSR_TVEC, a2
```

2. 之后开启MMU，加载一个临时的、不包含恒等映射的“蹦床”页表 (trampoline_pg_dir) 并开启分页。这部分与我们的部分相同。
3. 在此之后，会触发可控异常，CPU 尝试获取下一条指令时，因“蹦床”页表中没有对应的恒等映射，必然会触发一个指令页错误 (Page Fault)。但关键是，这个自陷阱法就是利用了这个中断，来实现它下一步的重定位。
4. 该异常使 CPU 立即放弃当前执行流，并根据 stvec 的设置，直接跳转到目标代码的虚拟地址上继续执行。

完成 Task 1 后，请你分析这两种重定位方式是否都需要恒等映射？为什么？

我们的实验必须要实现恒等映射，具体原因可以见下方 **双重映射的关键作用**部分的描述

Linux内核源码则不需要，它采用自陷阱法。它建立的 trampoline_pg_dir 故意不包含 csrw satp 之后指令的恒等映射。进而通过缺页触发异常，而它已经把异常处理的地址 (stvec) 设置成了它想去的虚拟地址实现地址空间切换。

2.3 Task 1：大页表双重映射和重定位

该部分由队友完成，简单介绍一下。

2.3.1 early_pg_dir 的设计思想

简单来说，就是我们需要开启分页来运行在高位虚拟地址的C函数，但创建精细页表本身就需要调用这些函数。

所以创建了一个临时的、粗粒度的“脚手架”页表 early_pg_dir。它采用 1GB 的大页 (Gigapage) 进行映射，优点是结构极其简单，只需填充 L2 根页表的两个表项即可，无需动态内存分配，可以在内核最早期被快速构建。

2.3.2 双重映射的关键作用

1. 恒等映射：0x80... (VA) → 0x80... (PA)

   在 head.S 中执行 csrw satp, a0 指令后，MMU 立即生效。
   此时，PC 寄存器中的值仍然是 0x80... 区域的地址。如果没有恒等映射，CPU 会将这个 PC 值当作虚拟地址去查询页表，因查无此项而立即触发指令页错误 (Instruction Page Fault)，导致系统崩溃。
   恒等映射保证了在切换瞬间，内核依然能继续执行后续的 relocate 指令。

2. 内核映射：0xffff... (VA) → 0x80... (PA)

   relocate 函数的最终目标是把 PC 指针跳转到高位虚拟地址。内核映射提前使得这些 0xffff... 的高位地址变得有效，并正确指向了存放内核代码的物理内存。如果没有内核映射，relocate 函数在最后执行 ret 跳转到高位地址时，同样会因地址非法而触发页错误。

2.3.3 relocate 函数的实现

relocate 函数通过以下步骤，将执行上下文从物理地址域切换到虚拟地址域：

1. 加载一个固定的偏移量 PA2VA_OFFSET (0xffffffd600000000 - 0x80000000)。
2. 将此偏移量加到关键寄存器上：
   (a) ra (返回地址寄存器): call relocate 的返回地址（即 mm_init 的地址）被修正为高位虚拟地址。
   (b) sp (栈指针寄存器): 内核栈的地址同样被修正到虚拟地址空间。
   (c) stvec (中断向量基址寄存器): 异常处理程序的入口地址也必须被修正。
3. 执行 ret 指令。该指令会将 ra 中已被修正的高位虚拟地址加载到 PC 寄存器。至此，CPU 的指令流就彻底切换到了虚拟地址空间，成功完成重定位。

2.4 Task 2：多级页表

该部分由我完成。

2.4.1 swapper_pg_dir的设计优势

当内核成功运行在虚拟地址之上，我们便可以调用 Buddy System 等复杂的 C 函数来构建一个“最终”的页表 swapper_pg_dir。它相对于 early_pg_dir 有以下核心改进：

1. **精细粒度**：使用 4KB 的标准页面进行映射，为内存提供最精细的控制。
2. **权限分离**：遵循最小权限原则，为内核的不同逻辑段赋予严格的访问权限，增强系统安全性。
3. **移除恒等映射**：过渡完成后，拆除“安全网”，使内核地址空间更加纯粹，避免混淆。

2.4.2 create_mapping 函数：

它模拟硬件 MMU 的行为来遍历并填充三级页表。

其核心逻辑是一个外层循环，每次处理一个 4KB 的页面映射。在循环内部，遵循一个固定的“检查-创建-获取”三步走流程：

1. **L2 → L1**: 使用 VA2VPN2 提取 L2 索引，在根页表中找到 L2 PTE。检查其 PTE_V (有效)位。若为0，则代表下一级 L1 页表不存在。此时调用 alloc_pages(1) 分配一页物理内存作为 L1 页表，将其物理页号（PPN）和 PTE_V 标志填入 L2 PTE。最后，从 L2 PTE 中解析出 L1 页表的地址。

```
uint64_t vpn2 = VA2VPN2(current_va);
uint64_t vpn1 = VA2VPN1(current_va);
uint64_t vpn0 = VA2VPN0(current_va);

uint64_t *l1_pgtbl; // 用来存放 L1 页表的(虚拟)地址
uint64_t *l0_pgtbl; // 用来存放 L0 页表的(虚拟)地址

// 2. 第一级查找：从 L2 (pgtbl) 找到 L1 页表
uint64_t *l2_pte_addr = &pgtbl[vpn2];

if (!(*l2_pte_addr & PTE_V)) { // 检查 V 位
    // L1 页表不存在！需要创建
    // (1) 分配一页物理内存
    void *new_page = alloc_pages(1); // 返回的是虚拟地址
```

```
        // (2) 清零
        memset(new_page, 0, PGSIZE);
        // (3) 在 L2 PTE 中填入 L1 页表的物理地址和标志位
        *l2_pte_addr = (PTE_PA2PPN(VA2PA(new_page)) | PTE_V);
    }
    // 现在，我们能保证 L1 页表一定存在了
    // 从 L2 PTE 中提取出 L1 页表的物理地址
    uint64_t l1_pgtbl_pa = ((*l2_pte_addr >> 10) << 12);
    // 再转换成虚拟地址
    l1_pgtbl = (uint64_t *)PA2VA(l1_pgtbl_pa);
```

2. **L1 → L0**：与L2 → L1相似，此处不赘述
3. **L0 → 物理页面**：使用 VA2VPN0 提取 L0 索引，找到最终的 L0 PTE。在此处，填入目标物理地址 pa 对应的 PPN，并与传入的 perm 权限（如 PTE_R | PTE_X）和 PTE_V 组合，形成一个“叶子节点”PTE。

在此过程中，通过检查 PTE 的 R,W,X 位来区分“树枝节点”和“叶子节点”。当 R=W=X=0 时，该 PTE 指向下一级页表；否则，它指向最终的物理页面。

2.4.3 setup_vm_final 函数：

它依据链接器脚本 vmlinux.lds 提供的符号，规划内核的内存布局，并调用 create_mapping 执行映射：

1. **映射 .text 段**: 从 _stext 到 _srodata，权限为 PTE_R | PTE_X (可读、可执行)。
2. **映射 .rodata 段**: 从 _srodata 到 _sdata，权限为 PTE_R (只读)。
3. **映射 .data 和 .bss 段**: 从 _sdata 到 _ekernel，权限为 PTE_R | PTE_W (可读、可写)。
4. **映射剩余物理内存**: 从 _ekernel 对应的物理地址之后，到整个物理内存末尾。这部分内存供 Buddy System 使用，权限为 PTE_R | PTE_W。

完成所有映射后，该函数计算 swapper_pg_dir 的物理地址，构造新的 satp 值并返回，为最终的页表切换做好准备。